



INGOT · IN PLAIN LANGUAGE

One product, one truth.

Why we're rebuilding the way our catalog decides what a product *is* — and how we know the new way is safe.

THE PROBLEM

The same product arrives three different ways.

SUPPLIER A SAYS

**Panadol Osteo Caplets
665mg 96**

weight: 35

barcode: ...4134

SUPPLIER B SAYS

**PANADOL OSTEO CPLT
665MG 96**

weight: 0.035 kg

barcode: ...4134

SUPPLIER C SAYS

**Panadol Osteo 665 mg (96
pack)**

weight: – not provided

barcode: ...4141 (different!)

We buy product data from many suppliers. They disagree, duplicate each other, and change their minds over time. **Something has to decide what's true — for hundreds of thousands of products.**

TODAY

The current system guesses — and forgets.



It picks a winner silently.

When suppliers disagree, one value wins by hidden rules. Nobody can answer a customer who asks *"why does it say this?"*



It overwrites the past.

Each update replaces what was there. *"What did this product say in March?"* is a question we simply cannot answer.



Every new country is a custom build.

Each market's product codes are hand-wired into the software. Expanding means months of engineering — every time.

THE PROBLEM — WORST CASE

One wrong link poisons a product. Today, there's no antidote.

A barcode gets coupled to the wrong product — it happens. From that moment, **every import flowing through that barcode** — photos, price, description — lands on the wrong record.

TODAY — IRREVERSIBLE

Which facts came in through the bad link? Unknown — imports overwrote fields in place, keeping no trace of the route.

Undo? Impossible. The fix is manual archaeology, field by field, hoping you found everything.

INGOT — ONE CORRECTION, CLEAN SWEEP

Every fact is stamped with the code it arrived through. Detach the wrong code, and every fact it carried leaves with it — automatically.

Records recompute from the corrected evidence, and out comes an exact list of everything that changed.

In ingot, a wrong coupling is **one correction plus a recomputation — with a printable list of everything it touched.** Today it's an archaeology project with no guarantee of completeness.

Keep everything. Like an accountant.

OLD WAY — ONE BOX, OVERWRITTEN

~~weight: 30 g~~
~~weight: 32 g~~
weight: 35 g ← only this survives

INGOT — A LEDGER, NEVER ERASED

Supplier A · Jan	30 g
Supplier B · Mar	32 g
Supplier A · Jun	35 g
Published answer	35 g — and we can show why

Accountants never erase an entry — they add new ones, and the balance is **calculated**. Ingot treats product facts the same way: every statement from every supplier is kept forever, and what we publish is calculated from them. So we can **explain any answer, rewind to any date, and recalculate everything** if a rule changes.

Identity that maintains itself.

SUPPLIER A CLAIMS

national 3612173 · barcode ...4577

SUPPLIER B CLAIMS

national 3612173

SUPPLIER C CLAIMS

barcode ...4577



ONE GOLDEN RECORD

Dafalgan 1g 60 tabs

national 3612173 · barcode ...4577

nobody assigned this — it's computed from what suppliers actually say



No manual stitching, ever.

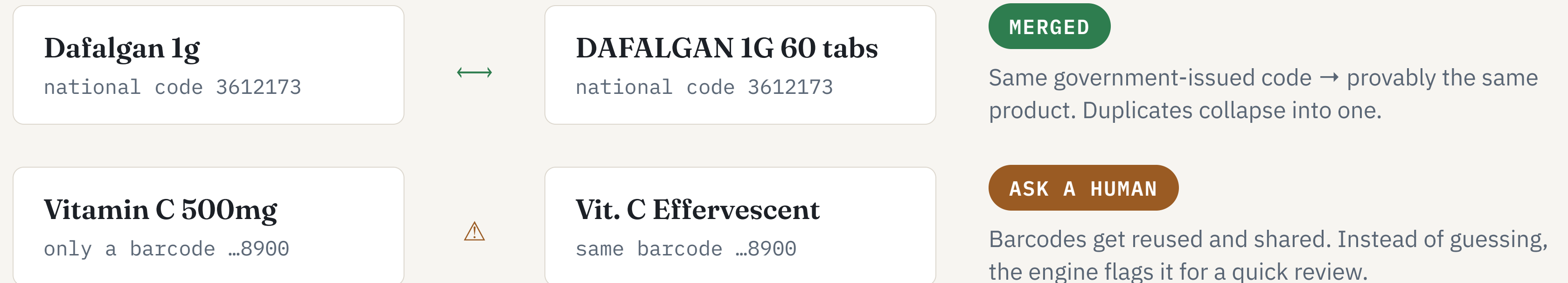
When a supplier adds, moves, or withdraws a code, the record follows automatically. The catalog heals itself instead of drifting.



Every code has a paper trail.

"Why is this barcode on this product?" always has an answer: who claimed it, from when until when.

Official codes decide what's the same product — with guardrails.



The old system trusted its own internal numbering. Ingot trusts **the codes printed on the actual product** — and when a code isn't trustworthy enough, it asks instead of guessing. **Wrong merges (the wrong photo on a medicine) become structurally impossible to do silently.**

The switch breaks nothing that exists today.

OLD INTERNAL NUMBER

#422156



UNCHANGED

Most of the catalog: same product, same record. Old links and integrations resolve exactly as before.

OLD INTERNAL NUMBERS

#118705 · #127851



MERGED

Duplicates disappear — but *both* old numbers keep pointing at the one combined record. Nobody's bookmark dies.

OLD INTERNAL NUMBER

#10701



REPLACED

A discontinued product doesn't go dark — its record names the successor, so customers get pointed forward.

Think postal forwarding after a move: **every old identifier resolves to the right new record, labeled with what happened.** Customers and integrations never notice the migration — they just get better answers.

PROOF

We don't switch on faith. We run both and compare.

Today's system keeps running, untouched		answers compared automatically		Ingot runs in the shadow, on the same data
617 automated checks run on every change	0 unexplained differences vs today's system on real product histories	4 markets already proven: BE, FR, books & Australia	127,000 Australian products added with configuration — not engineering	

Only when the side-by-side comparison stays clean do we flip the switch. **Migration by evidence, not hope.**

The same ledger works for everything in the catalog.

PHOTOS & VIDEOS

One image, every product it shows.

A photo carrying three barcodes reaches all three products. Duplicates collapse; orphaned images become visible instead of lost.

DESCRIPTIONS

Know whose words we publish.

Every text is tied to its supplier and language — and when two disagree, we can say exactly why one was chosen.

LEAFLETS & DOCUMENTS

The right leaflet on every pack.

Patient leaflets stay linked to each product they belong to — and old versions stay retrievable. That's compliance, built in.

INGREDIENTS

"Every product containing X" — one question.

Products connect to their substances, so recalls and ingredient checks stop being spreadsheet archaeology.

The engine already treats these as **first-class citizens** — same ledger, same guardrails, same history. The rollout should cover them from day one: **one system holding the whole catalog to one quality bar, instead of four separate ones.**

Two languages, one engine — compared honestly.

	Elixir THE REFERENCE	PHP OUR CURRENT STACK
FIT FOR THIS PROBLEM	Native — a language designed for recomputing answers from permanent logs	Capable — the engine is faithfully ported and fully verified
FITS TODAY'S PLATFORM & TEAM	New runtime to operate, new skills to hire	Yes — today's stack, run by today's team, zero adoption friction
WHERE IT SHINES	Designing & verifying: whole engine in one readable file, full check suite in under a second	Production inside the existing platform — no rewrite, no migration of the migration
THE ANSWERS IT GIVES	identical — byte for byte, proven by the same test suite passing in both	
NEW RISK INTRODUCED	Operational novelty (a stack we don't run yet)	None — it's the stack we already run

Use each where it's strongest: Elixir stays the reference where the design is built and verified fast; the PHP twin runs inside today's platform. And because both give identical answers, **the choice is reversible at any time — no lock-in either way.**

What this buys us.

- 1

New markets in days, not months.
A country is a configuration table. Australia already proved it.
- 2

Every answer is explainable.
"Why does it say this?" always has an answer: who said it, when, and why it won.
- 3

Errors become a short review list.
Conflicts get flagged for people instead of silently guessed — quality goes up, surprises go down.
- 4

Full history, forever.
Audit any product on any date. Nothing is ever lost again.

